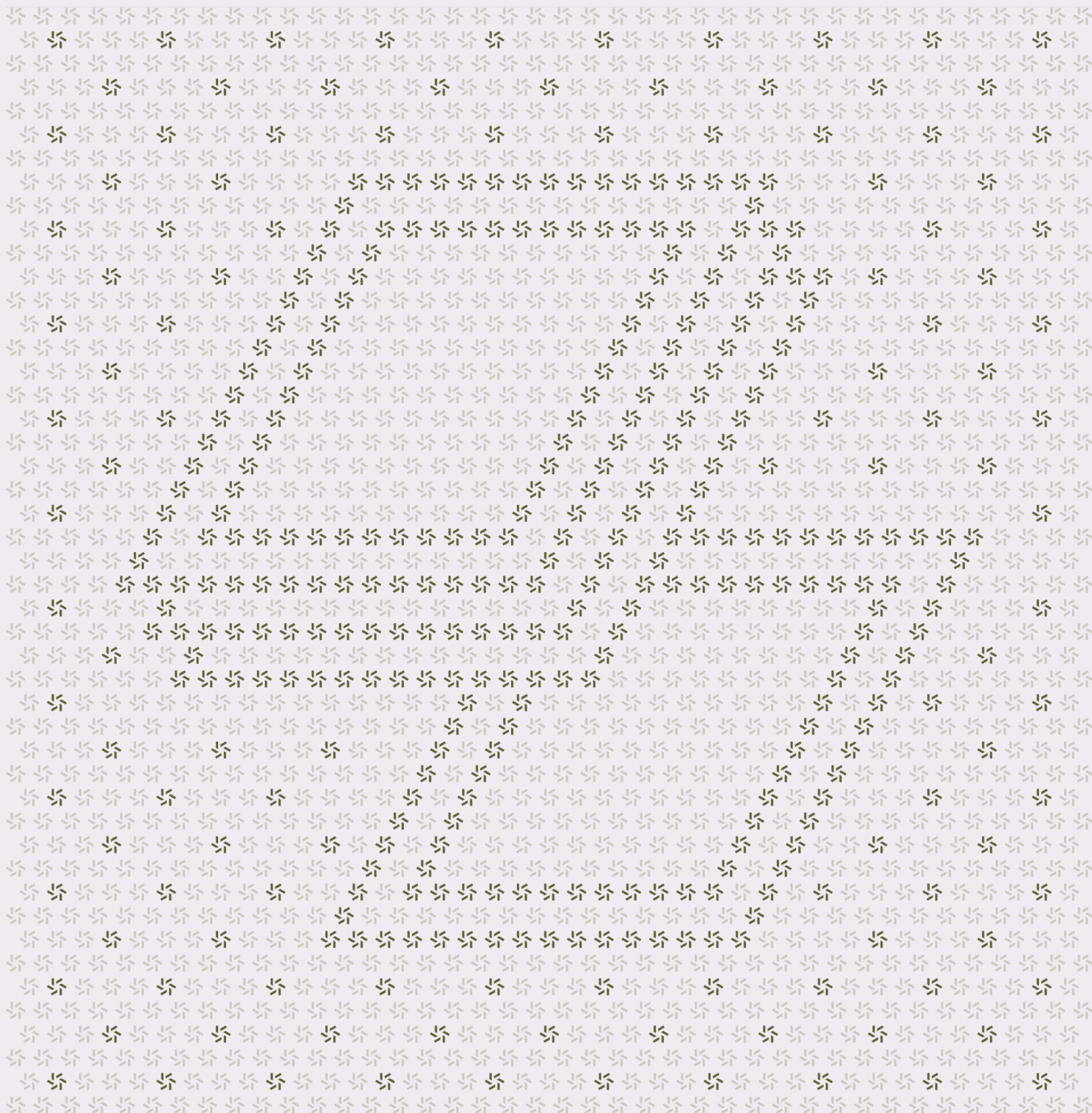


July 6, 2026

Zakura

Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	5
2. Introduction	7
2.1. About Zakura	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	10
2.5. Project Timeline	11
3. Detailed Findings	12
3.1. Body tip conflated as header frontier disables watchdog fallback	12
3.2. Repeated reconsiderblock panics the state write thread	14
3.3. Post-NU6.3 coinbase accepts non-empty Orchard actions	16
3.4. Header and body frontier desync can stall catch-up	18
3.5. Registry miss globally stalls sync dispatch	21
3.6. Ironwood cross-address error is unscored	23
3.7. Mempool rejects Ironwood empty-root anchor	25

4.	System Design	27
4.1.	Component: Ironwood V6 Transaction Verification	27
4.2.	Component: Ironwood Shielded Pool State	29
4.3.	Component: NU6.3 Activation and Version Transition	30
4.4.	Component: Zakura Native Block Sync and Frontier Tracking	32
4.5.	Component: Non-finalized State Mutation via invalidate and reconsider	34
4.6.	Component: Peer Misbehavior Scoring and Recovery	35

5.	Assessment Results	38
5.1.	Disclaimer	38

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Valar Group from June 23rd to July 6th, 2026. During this engagement, Zellic reviewed Zakura's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Is Ironwood correctly separated from Orchard across consensus and state logic?
 - Are Ironwood value balance, anchor, nullifier, and tree checks enforced consistently?
 - Can Zakura sync or frontier tracking get stuck or diverge during bootstrap or fork transitions?
 - Can invalid Ironwood or Zakura data bypass rejection, scoring, or recovery logic?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Problems caused by out-of-scope code
- Problems caused by internal configuration errors
- Threat models where the attacker is an internal administrator

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped Zakura targets, we discovered seven findings. No critical issues were found. Four findings were of high impact, one was of medium impact, and two were of low impact.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	4
Medium	1
Low	2
Informational	0



2. Introduction

2.1. About Zakura

Valar Group contributed the following description of Zakura:

Zakura is a fork of Zebra. Zebra is Zcash's now main consensus full node implementation. This audit scope covers the addition of the Ironwood shielded pool into Zakura, ensuring that the consensus rules follow what is required for the goal of adding a new shielded pool Ironwood, and disabling cross-address + coinbase transactions in Orchard

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the targets.

Architecture risks. This encompasses potential hazards originating from the blueprint of a system, which involves its core validation mechanism and other architecturally significant constituents influencing the system's fundamental security attributes, presumptions, trust mode, and design.

Arithmetic issues. This includes but is not limited to integer overflows and underflows, floating-point associativity issues, loss of precision, and unfavorable integer rounding.

Implementation risks. This encompasses risks linked to translating a system's specification into practical code. Constructing a custom system involves developing intricate on-chain and off-chain elements while accommodating the idiosyncrasies and challenges presented by distinct programming languages, frameworks, and execution environments.

Availability. Denial-of-service attacks are another leading issue in custom systems. Issues including but not limited to unhandled panics, unbounded computations, and incorrect error handling can potentially lead to consensus failures.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

Zakura Targets

Type	Rust
Platform	Zcash
Target	zakura
Repository	https://github.com/zakura-core/zakura
Version	abf68448a4bce61ff4ddae032bd6afd8e40b0458
Programs	<pre> xtask/src/main.rs zebra-chain/src/{block,history_tree,ironwood,lib,local_genesis, transaction,value_balance}.rs zebra-chain/src/block/commitment.rs zebra-chain/src/orchard/shielded_data.rs zebra-chain/src/parallel/tree.rs zebra-chain/src/parameters/{network,network_upgrade}.rs zebra-chain/src/parameters/network/{subsidy,testnet}.rs zebra-chain/src/primitives/{zcash_history,zcash_note_encryption, zcash_primitives}.rs zebra-chain/src/transaction/{hash,serialize,sighash,txid, unmined}.rs zebra-chain/src/transaction/unmined/zip317.rs zebra-consensus/src/{block,error,transaction}.rs zebra-consensus/src/block/check.rs zebra-consensus/src/primitives/halo2.rs zebra-consensus/src/transaction/check.rs zebra-network/src/protocol/external/codec.rs zebra-rpc/src/methods.rs zebra-rpc/src/methods/types/get_block_template.rs zebra-rpc/src/methods/types/get_block_template/{proposal, zip317}.rs zebra-state/src/{constants,error,request,response,service}.rs zebra-state/src/service/{finalized_state,non_finalized_state, read}.rs zebra-state/src/service/check/{anchors,nullifier}.rs zebra-state/src/service/finalized_state/{disk_db,zebra_db}.rs </pre>

```
zebra-state/src/service/finalized_state/disk_format/{chain,
upgrade}.rs
zebra-state/src/service/finalized_state/disk_format/upgrade/
{add_ironwood_tree,track_tx_locs_by_spends}.rs
zebra-state/src/service/finalized_state/zebra_db/{block,chain,
metrics,rollback,shielded}.rs
zebra-state/src/service/non_finalized_state/chain.rs
zebra-state/src/service/read/tree.rs
zebrad/src/components/sync.rs
zebrad/src/components/mempool/storage.rs
zebrad/src/components/mempool/storage/verified_set.rs
zebrad/src/components/zcashd_compat/managed.rs
```

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of three person-weeks. The assessment was conducted by two consultants over the course of two calendar weeks.

Contact Information

The following project manager was associated with the engagement:

Chad McDonald
 Engagement Manager
chad@zellic.io

The following consultants were engaged to conduct the assessment:

Gunhee Ahn
 Engineer
gunhee@zellic.io

Jie Ding
 Engineer
jie@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

June 23, 2026	Kick-off call
----------------------	---------------

June 23, 2026	Start of primary review period
----------------------	--------------------------------

July 6, 2026	End of primary review period
---------------------	------------------------------

3. Detailed Findings

3.1. Body tip conflated as header frontier disables watchdog fallback

Target	Zebra/Zakura sync state		
Category	Integration Risks	Severity	High
Likelihood	Medium	Impact	High

Description

The BestHeaderTip is used by the Zakura watchdog to decide whether native sync has a header frontier or should fall back to legacy sync. When no Zakura header frontier exists, Zebra reports the verified body tip as the best header tip instead of returning None.

The first fallback is in zebra-state/src/service/finalized_state/zebra_db/block.rs line 410, where the database returns the body tip when the Zakura header tip is absent. The second fallback is in zebra-state/src/service.rs line 1720, where the service wrapper also returns the block tip when the database has no header tip.

```
pub fn best_header_tip(&self) -> Option<(block::Height, block::Hash)> {
    let body_tip = self.tip();
    let zakura_header_tip = self.zakura_header_tip();

    match (body_tip, zakura_header_tip) {
        (Some(body_tip), Some(header_tip)) if body_tip.0 >= header_tip.0 =>
            Some(body_tip),
        (Some(_), Some(header_tip)) => Some(header_tip),
        (Some(body_tip), None) => Some(body_tip),
        (None, Some(header_tip)) => Some(header_tip),
        (None, None) => None,
    }
}
```

```
ReadRequest::BestHeaderTip => {
    let best_disk_header_tip = state.db.best_header_tip();
    let verified_block_tip = read::tip(state.latest_best_chain(), &state.db);

    Ok(ReadResponse::BestHeaderTip(
        match (best_disk_header_tip, verified_block_tip) {
            (Some(header_tip), Some(block_tip))
                if block_tip.0 > header_tip.0 => {
                    Some(block_tip)
                }
        }
    ))
}
```

```
        (Some(header_tip), _) => Some(header_tip),  
        (None, block_tip) => block_tip,  
    },  
))  
}
```

With default Zakura block sync, zebra calls `bootstrap_genesis_then_pause()` after genesis. The watchdog in `zebra/src/components/sync.rs` lines 378–389 then sees the header tip and verified tip at the same height, computes a gap of zero, and marks the node healthy. The intended no-frontier fallback branch is not reached.

Impact

A node that has committed genesis but has no Zakura header frontier can be treated as caught up. The legacy fallback does not fire, even though native Zakura sync has no header chain to follow.

Under eclipse or legacy-only peer conditions, the default configuration can leave the node stuck at genesis or another low verified body height until the frontier state is repaired or the node is restarted.

Recommendations

Make `BestHeaderTip` preserve the absence of a Zakura header frontier. If no Zakura header is stored, both `zebra_db` and the `ReadRequest::BestHeaderTip` service wrapper should return `None` rather than the body tip.

Add a regression test where the verified block tip is genesis and the Zakura header frontier is absent. The watchdog should treat this as no known frontier and allow the legacy fallback after the idle threshold.

Remediation

This issue has been acknowledged by Valar Group, and a fix was implemented in commit [bce912c5](#).

Valar Group provided the following comment:

This issue has been acknowledged by Valargroup as affecting the v2 p2p stack. Note, v2 p2p in the later production usage is turned off.

3.2. Repeated reconsiderblock panics the state write thread

Target	Zebra non-finalized state		
Category	Coding Mistakes	Severity	High
Likelihood	Medium	Impact	High

Description

reconsiderblock is intended to restore a block from invalidated_blocks and rebuild the corresponding non-finalized chain. The function removes the block from a clone of invalidated_blocks, not from the map itself. A second call on the same hash can therefore find the same invalidated entry again.

```
let invalidated_blocks = Arc::unwrap_or_clone(
    self.invalidated_blocks
        .clone()
        .shift_remove(height)
        .ok_or(ReconsiderError::MissingInvalidatedBlock(block_hash))?,
);
```

The rebuilt chain is inserted before the duplicate invalidated branch is filtered out. This can make the BTreeSet compare two chains with equal work and the same tip hash.

```
match self_hash.0.cmp(&other_hash.0) {
    Ordering::Equal => unreachable!("Chain tip block hashes are always
unique"),
    ordering => ordering,
}
```

Impact

A second reconsiderblock on the same block hash panics the state-write thread. This requires access to the reconsiderblock RPC method. After the panic, the node can no longer commit or finalize blocks until restart.

Recommendations

Remove the reconsidered entry from the real `invalidated_blocks` map before rebuilding the chain. The operation should become idempotent. A repeated call for the same hash should return `MissingInvalidatedBlock` instead of reaching `insert_with()`.

Add a regression test that invalidates a block, calls `reconsiderblock` once, then calls it again for the same hash. The second call should return a typed error and must not panic.

As defense in depth, harden the chain-set insertion path so that a duplicate chain tip is rejected before `Chain::cmp` reaches its `unreachable!` arm. The primary fix should make `reconsiderblock` idempotent, but the comparator should not be the only place where this invariant is enforced.

Remediation

This issue has been acknowledged by Valar Group, and a fix was implemented in commit [cbdd73f9](#).

Valar Group provided the following comment:

This bug was also present in upstream Zebra, and fixed with a similar patch.

3.3. Post-NU6.3 coinbase accepts non-empty Orchard actions

Target	Zebra consensus		
Category	Completeness	Severity	High
Likelihood	Low	Impact	High

Description

After NU6.3, coinbase transactions should not contain Orchard pool actions. ZIP-229 defines the V6 and Ironwood transaction format. ZIP-258 deploys NU6.3 and specifies that a coinbase transaction MUST NOT contain Orchard-pool Actions. Zebra enforces the older coinbase rules but does not reject a non-empty zero-value Orchard component.

The coinbase check rejects Orchard spends but does not check whether Orchard actions are present.

```
if let Some(orchard_shielded_data) = tx.orchard_shielded_data() {
    if orchard_shielded_data.flags.contains(Flags::ENABLE_SPENDS) {
        return Err(TransactionError::CoinbaseHasEnableSpendsWithOrchard);
    }
}
```

The NU6.3 Orchard freeze check only rejects value added to the Orchard pool.

```
let value_balance_orchard = tx
    .orchard_shielded_data()
    .map(|shielded_data| shielded_data.value_balance)
    .unwrap_or_else(Amount::<NegativeAllowed>::zero);

if height >= nu6_3_activation_height && value_balance_orchard < zero {
    return Err(TransactionError::DisabledAddToOrchardPool);
}
```

The payload uses a V5 transaction at NU6.3, which Zebra still routes through the V5 verifier.

Impact

A block producer can build a NU6.3 coinbase transaction with one zero-value Orchard action. Zebra accepts the proposal after Merkle root and block commitment recomputation, even though nodes enforcing the post-NU6.3 coinbase Orchard rule should reject it.

This can split Zebra from implementations that reject non-empty Orchard coinbase components after the Orchard to Ironwood transition.

Recommendations

Add an explicit post-NU6.3 coinbase rule that rejects any non-empty Orchard component, independent of `ENABLE_SPENDS` and `valueBalanceOrchard`.

Add a regression test at a NU6.3 activation height with a V5 coinbase that has one zero-value Orchard action. The transaction should be rejected before block proposal validation succeeds.

Remediation

This issue has been acknowledged by Valar Group, and a fix was implemented in commit [d402f4fa](#).

Valar Group provided the following comment:

At time of audit-handoff the spec for Ironwood still had ambiguity on this item, but at time of audit it was clear. The patch was made at time of spec clarity.

3.4. Header and body frontier desync can stall catch-up

Target	Zebra/Zakura sync state		
Category	Integration Risks	Severity	High
Likelihood	Medium	Impact	High

Description

Zakura stores header and body progress separately, but later sync logic assumes both frontiers describe the same parent-linked chain. That assumption is not always enforced.

The best-chain commit path can seed a Zakura header from a committed block by height without proving that the previous stored Zakura header is its parent.

```
if should_seed_zakura_header_from_non_finalized_commit(
    *seed_zakura_header_from_best_chain_commits,
    non_finalized_state,
    child_height,
    child_hash,
) {
    seed_zakura_header_from_committed_block(
        &finalized_state.db,
        child_height,
        &child_block,
    );
}
```

The database write path then handles same-height conflicts and inserts the new header, but does not compare the new header's parent hash against the stored header at the previous height.

```
if existing_zakura_header.is_some_and(|existing_header| existing_header !=
    block.header) {
    let best_header_tip: Option<(block::Height, block::Hash)> =
        db.zs_last_key_value(&zakura_hash_by_height);

    if let Some((best_header_tip, _)) = best_header_tip {
        for old_height in height.0..=best_header_tip.0 {
            let old_height = block::Height(old_height);

            // Truncates same-height conflicts and descendants, but does not
            // check block.header.previous_block_hash against height - 1.
```

```
// ...

if old_height != height
  && db.zs_contains(
    &tx_by_loc,
    &TransactionLocation::min_for_height(old_height),
  )
{
  return Err(CommitHeaderRangeError::ConflictingFullBlockHeader
{
  height: old_height,
});
}

// ...
}
}

self.zs_insert(&zakura_header_by_height, height, &block.header);
self.zs_insert(&zakura_hash_by_height, height, hash);
self.zs_insert(&zakura_height_by_hash, hash, height);
```

The body-gap path also starts from the verified body tip height. If the best header chain diverges below that height, required fork ancestor bodies can be skipped.

```
let start = verified_block_tip
  .and_then(|tip| tip.next().ok())
  .map_or(from, |first_missing| first_missing.max(from));

if start > best_header_tip {
  return Vec::new();
}
```

Impact

A node can know about a higher-work Zakura header chain but fail to request or commit the bodies required to follow it. Catch-up can stall until the inconsistent header/body frontier state is repaired.

Recommendations

When seeding or committing Zakura headers, verify parent-link continuity against the stored header at the previous height. If the parent does not match, truncate or repair the affected header

range before publishing the new frontier.

For body discovery, compute the common ancestor between the verified body chain and the best header chain. Missing-body queries should start after that ancestor, not after the current body tip height.

Remediation

This issue has been acknowledged by Valar Group, and a fix was implemented in commit [8551e3f9](#) ↗.

3.5. Registry miss globally stalls sync dispatch

Target	Zebra sync		
Category	Integration Risks	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

`sync_round()` gives registry-missed hashes global head-of-line priority. While any hash is waiting in `registry_miss_retry`, Zebra stops dispatching the remaining reserve hashes, even if those hashes are unrelated and available from peers.

```
let head_of_line_starved = !self.registry_miss_retry.is_empty();

if !past_lookahead && !head_of_line_starved && !reserve.is_empty() {
    debug!(
        tips.len = self.prospective_tips.len(),
        in_flight = self.downloads.in_flight(),
        reserve = reserve.len(),
        lookahead_limit,
        state_tip = ?self.latest_chain_tip.best_tip_height(),
        "requesting more blocks",
    );

    let response = timeout(
        BLOCK_VERIFY_TIMEOUT,
        self.request_blocks(std::mem::take(&mut reserve)),
    )
    .await
    .map_err(Report::from)?;
    reserve = Self::handle_hash_response(response)?;
    last_progress = Instant::now();
    continue;
}
```

`NotFoundKind::Registry` stores the missing hash for retry. The current retry budget is 60 attempts with a two-second backoff.

```
NotFoundKind::Registry => {
    metrics::counter!("sync.missing.block.registry.miss.count").increment(1);
}
```

```
let retry_count =
self.registry_miss_retry_counts.entry(hash).or_default();

if *retry_count < MISSING_BLOCK_REGISTRY_RETRY_LIMIT {
    *retry_count += 1;

    self.registry_miss_retry.insert(
        hash,
        tokio::time::Instant::now() + REGISTRY_MISS_RETRY_BACKOFF,
    );

    return Ok(());
}

self.registry_miss_retry_counts.remove(&hash);
self.registry_miss_retry.remove(&hash);
```

Impact

A peer that influences block hash ordering can place unavailable hashes before useful hashes. One unavailable window can delay dispatch of unrelated reserve hashes for roughly two minutes before the sync round restarts.

Recommendations

Do not make one registry-missed hash block dispatch for the whole reserve. Track the unavailable hash separately and continue dispatching unrelated hashes within the normal lookahead limit.

Add a regression test where one required hash is registry-missed while later reserve hashes are available. Zebra should retry the missed hash without blocking dispatch of the available reserve.

Remediation

This issue has been acknowledged by Valar Group, and is also present in Zebra.

3.6. Ironwood cross-address error is unscored

Target	Zebra consensus scoring		
Category	Code Maturity	Severity	Low
Likelihood	Medium	Impact	Low

Description

The transaction error `IronwoodDoesNotHaveEnableCrossAddress` is not included in the misbehavior score table. A peer can repeatedly send this invalid transaction class without being penalized through this misbehavior score path.

Sibling flag errors are scored at 100.

```
match self {
  // ...
  DisabledAddToOrchardPool
  | NotEnoughFlags
  | NotEnoughIronwoodFlags
  | OrchardHasEnableCrossAddress
  | WrongConsensusBranchId
  | MissingConsensusBranchId
  | LockedUntilAfterBlockHeight(_)
  | LockedUntilAfterBlockTime(_) => 100,
  _other => 0,
}
```

The missing Ironwood error falls through to the default score.

```
#[error("must have at least enable spend or enable output flag set")]
NotEnoughIronwoodFlags,

#[error("Orchard transactions MUST NOT have the EnableCrossAddress flag set")]
OrchardHasEnableCrossAddress,

#[error("Ironwood transactions MUST have the EnableCrossAddress flag set")]
IronwoodDoesNotHaveEnableCrossAddress,
```

Impact

Invalid Ironwood transactions can consume mempool or block validation work without this misbehavior penalty. The block path delegates transaction scoring to the same function, so the gap applies there as well.

Recommendations

Add `IronwoodDoesNotHaveEnableCrossAddress` to the same scoring group as the sibling flag errors. Add a regression test that checks its mempool and block misbehavior score.

Remediation

This issue has been acknowledged by Valar Group, and a fix was implemented in commit [2bbda9c6](#).

Valar Group provided the following comment:

This error was removed entirely. At time of audit hand-off this rule had ambiguity of whether Ironwood must enable cross address or its client choosable. Now the consensus rule is to not enforce this, thus there is no peer misbehavior. Related p2p errors are now included in scoring though.

3.7. Mempool rejects Ironwood empty-root anchor

Target	Zebra state anchor checks		
Category	Completeness	Severity	Low
Likelihood	Low	Impact	Low

Description

A V6 transaction can validly use the Ironwood activation empty root as its `shared_anchor`. In the mempool path, if there is no non-finalized best chain, Zebra checks only the finalized Ironwood anchor column family. That finalized column family does not contain the activation empty root until the activation height is reached.

```

if !parent_chain
  .map(|chain| {
    chain
      .ironwood_anchors
      .contains(&ironwood_shielded_data.shared_anchor)
  })
  .unwrap_or(false)
  && !finalized_state.contains_ironwood_anchor
    (&ironwood_shielded_data.shared_anchor)
{
  return Err(ValidateContextError::UnknownIronwoodAnchor {
    anchor: ironwood_shielded_data.shared_anchor,
    height,
    tx_index_in_block,
    transaction_hash,
  });
}

```

The block path has the empty root because `Chain::new` initializes non-finalized Ironwood anchors. This makes the mempool and block paths disagree at the activation boundary.

Impact

The mempool can reject a valid V6 transaction as `UnknownIronwoodAnchor` before a non-finalized best chain exists, while the same transaction is accepted in the block path. The issue clears once the chain reaches the activation height.

Recommendations

Initialize the finalized Ironwood anchor set with the empty root before activation, matching Sapling and Orchard behavior. Alternatively, make the mempool anchor check explicitly accept the Ironwood empty root at activation when no non-finalized parent chain exists.

Add a regression test for a V6 transaction whose Ironwood shared_anchor is the activation empty root with parent_chain = None.

Remediation

This issue has been acknowledged by Valar Group, and a fix was implemented in commit [c45fe95f](#).

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Component: Ironwood V6 Transaction Verification

Description

This component decides whether a NU6.3 V6 transaction is consensus-valid before its effects reach pool state and the fee calculation. The verifier serves both the block and mempool paths, and splits into a synchronous quick-check phase and an async proof-and-state phase.

The quick-check phase runs cheap synchronous checks in a fixed order, before any UTXO load or proof work:

1. `has_inputs_and_outputs`, then `has_enough_orchard_flags` and `has_enough_ironwood_flags`, which require each present shielded bundle to set at least one action flag.
2. The cross-address flag rules, which separate the two pools by flag. `orchard_cross_address_disabled` rejects a transactional Orchard bundle that sets `ENABLE_CROSS_ADDRESS` (bit 2 of the NU6.3 flag format), and `ironwood_cross_address_enabled` rejects an Ironwood bundle that does not set it.
3. `consensus_branch_id`, then `shielded_proof_size_is_canonical`, which under the NU6.2-onward rule rejects an Orchard or Ironwood bundle whose proof is not the canonical length for its action count (GHSA-jfw5-j458-pfv6), so trailing padding cannot be smuggled at a canonical-sized fee.
4. Coinbase rules. A coinbase is rejected outright in the mempool; otherwise `coinbase_tx_no_prevout_joinsplit_spend` rejects a coinbase whose Orchard or Ironwood bundle sets `ENABLE_SPENDS`, alongside the JoinSplit and Sapling-spend bans.
5. `spend_conflicts`, which rejects a transaction that reveals the same Ironwood nullifier twice within itself, next to the Sprout, Sapling, and Orchard duplicate checks.

The async phase loads spent UTXOs, binds the transaction to the height's network upgrade, and routes on version. Versions below V4 are rejected at the version match, V4 and V5 use their existing verifiers, and V6 routes to `verify_v6_transaction`, which first calls `verify_v6_transaction_network_upgrade`. That gate accepts only when both the active upgrade and the transaction's declared upgrade are Nu6_3. It then assembles the bundle checks:

1. `verify_transparent_inputs_and_outputs` feeds each transparent input to the script verifier; a coinbase has none.

2. `verify_sapling_bundle` queues the Sapling bundle.
3. `verify_v6_orchard_bundle` queues the transactional Orchard bundle to `v6_verifier`.
4. `verify_ironwood_bundle` queues the Ironwood bundle, delegating to `verify_v6_orchard_bundle`, so it reaches the same `v6_verifier` under one shared V6/Ironwood verifying key. At NU6.3 the transactional Orchard bundle and the Ironwood bundle share this one V6 action circuit; the two pools are separated by note-commitment and nullifier state, not by circuit.

Each bundle is queued to the V6 batch verifier, which checks the Halo2 action proof and the binding signature together. The verifier checks the bundle value-balance relation, so a bundle passes only if its proof, signature, and value balance are mutually consistent. The verifying key is fixed by the verifier the code selects, never by an attacker-supplied field, so a V6 proof is never checked against the V5 key. On the mempool path a best-chain-tip nullifier-and-anchor query is also appended, which asserts the transaction's anchors are known and its nullifiers unrevealed against the best chain tip.

Test coverage

Cases covered

- V6 txid and auth-digest commitment vectors, and rk-identity rejection, in `zebra-consensus/src/transaction/tests.rs`.
- Misbehavior scoring for the Ironwood cross-address rejection, added as a regression during this review.

Cases not covered

- Arbitrary for Transaction has no `v6_strategy`, so `proptest` never generates a V6 transaction and no property test exercises the V6 path.
- Ironwood flag combinations beyond the fixed vectors, for example `ENABLE_OUTPUTS` set without `ENABLE_SPENDS`.
- Coinbase carrying an Ironwood bundle with `ENABLE_SPENDS` set, and coinbase acceptance of a zero-value Ironwood or Orchard action.
- A single V6 transaction carrying both a transactional Orchard bundle and an Ironwood bundle, exercising both `v6_verifier` queues at once.

Attack surface

- **V6 wire payload.** A transaction author controls the V6 body, including flags, proof bytes, value balance, actions, anchors, and nullifiers. The boundary is that these fields must be fully bound by proof, signature, flag, and state checks before they affect pool state or fees.
- **Flag format.** The cross-address bit separates transactional Orchard from Ironwood. The expected invariant is that both pools reject the opposite flag state and apply

equivalent recovery behavior. Finding `ironwood-error-unscored` came from the rejection side being present while the `peer-score` side was missing.

- **Circuit selection.** The verifying key is chosen by transaction version and active upgrade, not by an attacker-supplied field, so a V6 bundle cannot be steered onto a weaker historical key.
- **Nullifier reuse.** `spend_conflicts` rejects reuse of one Ironwood nullifier within a single transaction. Reuse across transactions and blocks is enforced later by pool state, not in this component.

4.2. Component: Ironwood Shielded Pool State

Description

This component holds and enforces the Ironwood note-commitment tree, its anchors, and its nullifier set, kept separate from Orchard and Sapling across finalized and non-finalized state and reorgs. The state lives in two halves, the non-finalized Chain and dedicated finalized column families.

A non-finalized Chain holds its anchors in an `ironwood_anchors` set with a by-height index, and its nullifiers in an `ironwood_nullifiers` map. As a block commits, `add_ironwood_tree_and_anchor` records the new tree root, from both `Chain::new` and the per-block update path, and a matching `remove` reverses it on a reorg pop. Nullifier uniqueness within a chain is enforced by `add_ironwood_to_non_finalized_chain_unique`, which rejects a duplicate insert. The finalized state keeps the same anchors and nullifiers in their own column families, and at the NU6.3 activation height it writes the empty tree and its anchor even with no shielded activity.

Because `ironwood::Nullifier` reuses the Orchard nullifier representation, the state logic cannot rely on the Rust type alone to distinguish pool context. The dedicated `DuplicateIronwoodNullifier` error keeps Ironwood failures separate from Orchard failures. Uniqueness is checked against all three layers: a committed block against finalized state, a mempool transaction against non-finalized and finalized state, and the transaction against itself.

A transaction's `shared_anchor` is checked by `sapling_orchard_ironwood_anchors_refer_to_final_treestates`, which consults the non-finalized chain's anchors first and the finalized anchor set second, and rejects with `UnknownIronwoodAnchor` only if both miss. The activation boundary is important because the empty Ironwood root can be present in a freshly built non-finalized Chain before it is visible in the finalized anchor set. Finding `empty-root-anchor` came from a mempool path that had no best chain and therefore fell back to finalized anchors only, while the block path checked against a newly built chain that had seeded the empty root.

Test coverage

Cases covered

- `ironwood_empty_root_visible_in_chain_but_not_finalized_cf_audit`, added during this review, asserting the empty root is present in a freshly built Chain but absent from the finalized CF.
- Rollback coverage in `finalized_state/tests/rollback.rs`: the empty activation tree and its indexed anchor survive rollback, and Ironwood nullifiers and subtrees above the target are pruned.
- `reject_duplicate_ironwood_nullifiers_in_chain` in `check/tests/nullifier.rs`, expecting `DuplicateIronwoodNullifier`.

Cases not covered

- `check/tests/anchors.rs` tests only Sprout and Sapling anchors; there is no Ironwood or Orchard anchor test, so block-versus-mempool `shared_anchor` agreement is not exercised there.
- Mempool-versus-block anchor agreement at `activation - 1` with no best chain (finding: empty-root-anchor).
- Deep-reorg symmetry of Ironwood anchor and nullifier state under repeated push and pop.

Attack surface

- **shared_anchor.** A transaction author controls the transaction's `shared_anchor`. The expected invariant is that the block and mempool paths consult equivalent Ironwood anchor sets for the same chain state. Finding `empty-root-anchor` came from an activation-boundary mismatch in that invariant.
- **Nullifiers.** A transaction author controls the revealed Ironwood nullifiers. Uniqueness is enforced across the transaction, the non-finalized chain, and the finalized state, with a dedicated `DuplicateIronwoodNullifier` error so an Ironwood collision is not masked by the Orchard path.
- **Pool separation.** Ironwood tree, anchor, and nullifier state is disjoint from Orchard and Sapling in both the non-finalized chain and the finalized state. We did not identify a direct Orchard/Ironwood state-aliasing issue in this boundary.

4.3. Component: NU6.3 Activation and Version Transition

Description

This component covers the NU6.3 activation boundary: resolving the consensus branch ID, binding blocks and transactions to the upgrade active at their height, gating V5 and V6 transactions, and applying the post-NU6.3 coinbase rules.

`Nu6_3` is a `NetworkUpgrade` variant ordered by activation height between `Nu6_2` and `Nu7`. In the scoped snapshot, its production branch ID and public-network activation heights were not final. NU6.3-gated code therefore depends on the deployment configuration before these rules become live on production networks. We treated this as an activation-boundary risk rather than a

standalone finding.

The active upgrade at a height is `NetworkUpgrade::current`, the last activation entry at or below the height. A V5 or V6 transaction carries its own `network_upgrade`, derived from `nConsensusBranchId`. `consensus_branch_id` binds the two: for a version-5-or-later transaction at NU5 or later, the declared upgrade must equal the active one, otherwise it is rejected as `WrongConsensusBranchId` or `MissingConsensusBranchId`. The block-level `check_transaction_network_upgrade_consistency`, reached from `merkle_root_validity`, applies the same rule and returns `WrongTransactionConsensusBranchId`.

Version gating is enforced by the per-version verifiers.

`verify_v5_transaction_network_upgrade` admits V5 from NU5 through NU6.3 and NU7.

`verify_v6_transaction_network_upgrade` is strict: it accepts only when both the active upgrade and the transaction's declared upgrade are `Nu6_3`, so a V6 transaction is rejected outside NU6.3, including at NU7, and versions below V4 are rejected at the version match.

The post-NU6.3 coinbase rules are applied by `coinbase_tx_no_prevout_joinsplit_spend`. For a coinbase it rejects JoinSplits, Sapling spends, and any Orchard or Ironwood bundle whose flags set `ENABLE_SPENDS`. The relevant boundary for finding orchard-coinbase-actions is narrower: a block producer controls the coinbase Orchard component, while ZIP-258 requires that component to be empty after NU6.3. We do not claim an Ironwood component emptiness rule here.

Test coverage

Cases covered

- V5 and V6 height-and-upgrade gating vectors in `zebra-consensus/src/transaction/tests.rs`, and V6 rejection under a pre-NU6.3 branch ID in `zebra-chain`.
- `consensus_branch_id` mismatch coverage in `zebra-chain/src/transaction/tests/vectors.rs`.
- NU6.3 pool and flag rules: net-deposit rejection, Orchard cross-address disabled, Ironwood cross-address required, and migration balances.

Cases not covered

- Whether a post-NU6.3 coinbase may contain a zero-value Orchard action. The only related test, `v5_coinbase_transaction_without_enable_spend_flag_passes_validation`, inserts a single zero-value Orchard action into a V5 coinbase and asserts it passes, so the suite confirms the permissive behavior rather than pinning the intended rule (finding: orchard-coinbase-actions).
- Behavior at `activation - 1`, `activation`, and `activation + 1` across the block and mempool paths.

Attack surface

- **Coinbase Orchard component.** A block producer controls the coinbase Orchard component. The expected invariant is that activation rules reject Orchard actions in coinbase transactions after NU6.3. Finding orchard-coinbase-actions came from this boundary.
- **Declared consensus branch ID.** A transaction's network_upgrade is attacker-declared but must match the height's active upgrade, so a mismatched branch ID is rejected on both the transaction and block paths.
- **Activation parameters.** NU6.3-gated rules depend on the branch ID and activation heights supplied for the target network. Correctness of this boundary depends on those parameters being present before deployment.

4.4. Component: Zakura Native Block Sync and Frontier Tracking

Description

This component drives the Zakura v2_p2p header and body sync and decides when to abandon it for legacy sync. It tracks a header frontier and a verified body tip:

`bootstrap_genesis_then_pause` bootstraps genesis, then polls on a fixed interval and hands control to legacy sync if body sync stalls.

The watchdog decision is `zakura_block_sync_stalled`. It does not treat "the verified tip moved" as progress on its own. It computes the gap between the header frontier and the verified tip, and credits progress only when the node is within `ZAKURA_NEAR_TIP_GAP` (2 blocks) of the frontier, or the gap has closed by at least `ZAKURA_BLOCK_SYNC_MIN_CLOSURE` (64 blocks) since the last credited progress. Only when no header frontier is known does it fall back to the plain "verified height increased" rule. Idle polls accumulate over the `ZAKURA_BODY_SYNC_STALL_TIMEOUT` (five-minute) window, and once they reach the limit the watchdog trips and hands off to legacy sync.

The frontier comes from a best-header-tip read, documented as the highest header held on disk. It is computed at two layers, the database accessor `best_header_tip` and its service handler, and both substitute the verified body tip when no header frontier exists rather than reporting its absence: with a body tip present but no Zakura header row, each returns the body tip instead of None. A v2_p2p node with no recorded header row, for example one eclipsed at genesis, therefore reports a header frontier equal to its body tip. The watchdog then sees a zero gap, credits itself as caught up, and never falls back (finding: header-frontier-fallback).

Stored Zakura headers are written by `prepare_zakura_header_from_committed_block`, reached from the commit path under the runtime-enabled `enable_zakura_header_seed_from_committed_blocks`. It writes the header keyed by height, guarded only by same-height equality and conflict-truncation checks; it never verifies that the block's `previous_block_hash` matches the stored header at the preceding height. Neither state batch verifies that link: the sibling header-range path `prepare_header_range_batch` anchors to a known header and validates difficulty and time per header through `header_is_valid_for_recent_chain`, but it does not compare `previous_block_hash` either. The committed-block seed path thus persists the header frontier with no parent-link check, which lets

the stored header sequence desync from a parent-linked chain (finding: frontier-desync-stall).

Registry-missed hashes gate the whole reserve. In `sync_round`, dispatch of the queued reserve is skipped while any hash sits in the registry-miss retry map, so a single hash the peer set cannot supply, held there with a two-second backoff after a `NotFound`, suppresses dispatch of every unrelated available hash in the reserve until its backoff elapses (finding: registry-miss-stall).

Test coverage

Cases covered

- Watchdog rules in `zebrad/src/components/sync/tests/fallback.rs`: stall, gossip-trickle, near-tip, steady-progress, and the no-header-frontier legacy rule.
- `body_tip_as_header_frontier_suppresses_fallback_audit_high`, the regression showing the body-tip-as-frontier input never falls back while a correct `None` does, added during this review.
- `best_header_tip_reports_body_tip_without_header_frontier_audit_high` and `best_header_tip_reports_body_tip_with_zakura_seed_default_config_audit_high` in the state layer, added during this review.

Cases not covered

- Seed-time parent-link continuity of `prepare_zakura_header_from_committed_block`; no regression asserts that a non-parent-linked header is rejected (finding: frontier-desync-stall).
- Long eclipse or legacy-only bootstrap windows.
- Reserve dispatch progress while one hash is in registry-miss backoff (finding: registry-miss-stall).

Attack surface

- **Header and body responses.** A selected or eclipsing Zakura peer can influence which headers and bodies the victim sees during native sync. The expected invariant is that an absent header frontier remains distinguishable from a body tip. Finding `header-frontier-fallback` came from that distinction being lost.
- **Seeded headers.** Headers seeded from committed blocks become durable sync-frontier state. The expected invariant is that the stored header frontier remains parent-linked to the prior stored header. Finding `frontier-desync-stall` came from this link not being checked on the seed path.
- **Registry-missed hashes.** Peers influence which block hashes enter the sync reserve. The expected invariant is that failure to fetch one unavailable hash should not globally suppress dispatch of unrelated available hashes. Finding `registry-miss-stall` came from one retry entry gating the entire reserve.

4.5. Component: Non-finalized State Mutation via invalidate and reconsider

Description

This component applies the `invalidateblock` and `reconsiderblock` RPC operations to the non-finalized chain set. These methods require authorized RPC access. Both run on the single non-finalized write loop, so they mutate state on one thread and a panic here crashes the state-write task.

The chain set is an ordered set of chains, sorted by `Chain::cmp`, which compares by `partial_cumulative_work` first and then breaks ties on the tip block hash. Its final arm is an `unreachable!` reached when both are equal; the comment there states that the panic enforces the chain-set unique-chain invariant. Because `PartialEq` also routes through `Chain::cmp`, any equality comparison inside the set can reach that arm.

`invalidate_block` finds the chain containing the target hash. If the hash is that chain's root it removes the whole chain; otherwise it forks the chain at the hash, inserts the truncated fork, and drops the original. The removed subtree is stored in `invalidated_blocks`, a map keyed by the subtree root's height, bounded by count (`MAX_INVALIDATED_BLOCKS`, 100) and pruned below the finalized tip on finalize.

`reconsider_block` looks up the invalidated entry whose first block hash equals the target, rebuilds its parent chain (a fresh chain off the finalized tip, or the subtree's parent chain), pushes each stored block back, and re-inserts the rebuilt chain. It is not idempotent: it removes the entry from a clone of `invalidated_blocks` rather than from the map itself, so the real map still holds the entry after a successful reconsider, and the height pruning on this path removes only entries below the finalized tip. A second `reconsiderblock` on the same hash re-finds the stale entry and rebuilds a chain with the same `partial_cumulative_work` and tip hash as the chain the first call already inserted; inserting it into the set compares the two equal chains, reaches the `unreachable!`, and panics the write thread, halting further commits (finding: `reconsiderblock-panic`).

Test coverage

Cases covered

- Single-invalidate and invalidate-then-reconsider happy paths in `non_finalized_state/tests/vectors.rs`.
- `repeated_reconsider_same_invalidated_block_panics_audit_high`, a `#[should_panic]` regression for the double-reconsider panic, added during this review.
- `reconsider_block_leaves_stale_invalidated_entry_audit_high`, which isolates the clone-removal root cause without panicking, added during this review.

Cases not covered

- Interaction of `invalidated_blocks` retention with deep non-finalized state and repeated invalidate cycles at one height, where the `Height` key means a second invalidation overwrites the prior entry.

- Concurrent invalidate and reconsider of overlapping subtrees.

Attack surface

- **Repeated reconsiderblock.** An authorized RPC client can call `reconsiderblock` repeatedly on the same hash. The expected invariant is that reconsidering an already-restored block is idempotent or returns a recoverable error. Finding `reconsiderblock-panic` came from stale invalidation state allowing a duplicate chain to reach the chain-set comparator.
- **Chain-set uniqueness.** Correctness of the chain set depends on no two chains comparing equal under `Chain::cmp`. Because `PartialEq` also routes through `cmp`, any path that admits two equal chains turns a soft invariant break into a hard panic rather than a recoverable error.
- **Invalidated-blocks retention.** `invalidated_blocks` is bounded by `MAX_INVALIDATED_BLOCKS` and finalized-height pruning. We reviewed this as a resource-retention boundary, but did not treat it as a standalone finding.

4.6. Component: Peer Misbehavior Scoring and Recovery

Description

This component turns consensus and transaction errors into misbehavior scores that ban peers, and covers the separate Zakura block-sync reactor, which handles its own misbehavior signal on an independent path. The boundary is not just whether bad data is rejected, but whether rejection, disconnect, retry, and peer scoring remain aligned.

On the consensus scoring path, error variants map to a score.

`TransactionError::mempool_misbehavior_score` returns a flat 100 for a fixed list of variants and 0 for everything else, and `BlockError::misbehavior_score` delegates transaction errors to it. A nonzero score travels from the sync, mempool, and inbound producers over the misbehavior channel to a batcher, which accumulates per-peer scores and flushes them as `MetaAddr::new_misbehavior` changes. `AddressBook::update` bans a peer once its accumulated score reaches `MAX_PEER_MISBEHAVIOR_SCORE` (100). Because the increment for a scored error is a flat 100, equal to the threshold, any scored error is a single-strike ban.

The Ironwood cross-address rejection is not on that scored list.

`ironwood_cross_address_enabled` returns `IronwoodDoesNotHaveEnableCrossAddress` when an Ironwood bundle omits `ENABLE_CROSS_ADDRESS`, but that variant is absent from the `mempool_misbehavior_score` 100-arm, so it falls through to the zero default. Its two siblings are scored: the Orchard mirror `OrchardHasEnableCrossAddress` and the Ironwood-flag error `NotEnoughIronwoodFlags` each return 100. Finding `ironwood-error-unscored` came from this mismatch between a consensus rejection and peer-scoring behavior.

The Zakura block-sync reactor is the second path, and its misbehavior signal is kept out of the scoring and ban machinery. Its handlers classify serving-side and download-side violations into a `BlockSyncMisbehavior` value, and `report_misbehavior` forwards a

`BlockSyncAction::Misbehavior` and increments a violation metric. That action is consumed by `drive_block_sync_actions`, which logs it as a recorded violation; it never enters the misbehavior channel, so it neither scores nor bans a peer, and the registry keeps no per-peer misbehavior state. Disconnects are a separate concern: a frame that fails to decode returns `SinkReject::protocol`, which cancels the connection, so a malformed block-sync frame still drops the peer even though it accrues no ban score. Several stale comments in the block-sync code still describe misbehavior as scored or disconnect-driving, which no longer matches this record-only-plus-transport-cancel behavior.

Test coverage

Cases covered

- `ironwood_cross_address_error_unscored_audit` in `zebra-consensus/src/transaction/tests.rs`, added during this review, asserting `OrchardHasEnableCrossAddress` and `NotEnoughIronwoodFlags` score 100 while `IronwoodDoesNotHaveEnableCrossAddress` scores 0.
- Block and transaction misbehavior-score tables in `zebra-consensus/src/block/tests.rs`.
- Zakura reactor tests asserting a `BlockSyncAction::Misbehavior` is emitted for violations, and that a malformed frame reports `MalformedMessage` and cancels the connection.

Cases not covered

- Any assertion tying a nonzero score to the Ironwood cross-address rejection; the current test pins the zero (finding: `ironwood-error-unscored`).
- End-to-end accumulation to a ban across mixed Ironwood and Orchard violations from one peer.

Attack surface

- **Invalid but well-formed peer input.** A peer can send transactions or Zakura block-sync responses that are syntactically valid but semantically rejected. The expected invariant is that rejection paths have the intended recovery behavior, including disconnects or scoring where appropriate.
- **Unscored consensus rejection.** The Ironwood cross-address error maps to zero score while sibling flag errors map to 100. Finding `ironwood-error-unscored` came from this rejection path being added without the matching peer-score mapping.
- **Zakura block-sync path.** The Zakura block-sync misbehavior signal is logged as a recorded violation but never accrues ban score or feeds the address book. Disconnects come only from the transport, so a malformed frame can cancel the connection even though semantic block-sync violations are only recorded.
- **Single-strike scoring.** On the consensus path a scored error increments by a flat 100, equal to the ban threshold, so scoring is all-or-nothing, with no graduated accumulation

for borderline behavior.

5. Assessment Results

During our assessment on the scoped Zakura targets, we discovered seven findings. No critical issues were found. Four findings were of high impact, one was of medium impact, and two were of low impact.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.